

Frontend + AI + Hackathon Winning Playbook

Expanded field manual: UI/UX, design semantics, frontend libraries, APIs, backend, databases, AI inference, deployment, agentic coding, security, observability and a repeatable hackathon strategy.

Goal: maximize your odds of shipping a working, visually strong, technically credible demo under time pressure. This is a decision system for choosing the right tool quickly.

Volatile-data rule: free tiers, model names, quotas and pricing change frequently. Confirm current limits on the linked official docs before an event.

1. The default winning stack

For a web hackathon where speed matters, start with a boring backend and an excellent frontend. A strong default is:

Next.js / React
+ Tailwind CSS
+ shadcn/ui
+ Supabase (Postgres + Auth + Storage + Realtime)
+ one server-side API layer / Edge Function
+ Gemini / OpenRouter / NVIDIA / another provider
+ Vercel or Cloudflare
+ GitHub + CI
+ Sentry if time permits

This is a default, not a religion. Change the stack when the project constraint demands it.

2. UI/component ecosystem

Resource	Reference	Use / decision rule
Tailwind CSS	https://tailwindcss.com/	Utility-first styling; fast responsive UI and consistent design tokens.
shadcn/ui	https://ui.shadcn.com/	Source-owned React components; excellent hackathon default because you can modify them.
Radix Primitives	https://www.radix-ui.com/primitives	Accessible unstyled primitives for dialogs, menus, popovers, tabs and tooltips.
Headless UI	https://headlessui.com/	Unstyled accessible components that pair well with Tailwind.
Magic UI	https://magicui.design/	Animated/polished components for visually strong landing pages.
Aceternity UI	https://ui.aceternity.com/	Animated React/Tailwind components for standout marketing/portfolio sections.
daisyUI	https://daisyui.com/	Fast Tailwind component classes for conventional application UI.
Flowbite	https://flowbite.com/	Tailwind component ecosystem for common application patterns.
Uiverse	https://uiverse.io/	Community UI snippets for inspiration and rapid starting points.

Rule: use one primary component system. Mixing several component packs usually produces inconsistent spacing, radii and interaction behavior.

3. UI/UX semantics: choose the visual language by product meaning

I am interpreting your 'UCM semantics' request as practical semantic mapping of user, context, mood and product meaning to a visual system. Use this as a fast design-selection matrix.

Style	Semantic signal	Strong fit	Watch out
Glassmorphism / Liquid Glass	Premium, futuristic, transparent	AI assistants, fintech dashboards, media, creative tools, premium SaaS	Can destroy contrast/readability if overused.
Neobrutalism	Bold, direct, youthful	Developer tools, student products, hackathons, indie SaaS, portfolios	Weak fit for calm/institutional products.
Minimalism	Trust, clarity, efficiency	SaaS, productivity, B2B, documentation	Do not add decoration just to fill whitespace.
Skeuomorphism	Familiarity, physical metaphor	Music/creative tools, simulations, themed experiences	Metaphor can become visual clutter.
Claymorphism	Soft, friendly, playful	Education, consumer apps, playful dashboards	Can feel childish in serious enterprise UX.
Retro / Y2K	Nostalgia, experimentation	Creative portfolios, campaigns, games, experimental hackathons	Poor fit for conventional trust-heavy products.
Bento grid	Modular, feature-rich	AI platforms, product pages, portfolios, dashboards	Too many cards destroy hierarchy.
Asymmetric	Creative, editorial, premium	Portfolios, agencies, art, fashion, storytelling	Requires deliberate composition.

Fast rule: trust → minimalism; premium/futuristic → glass; playful → clay; rebellious/technical → neobrutalism; nostalgic → Y2K; feature-heavy → bento; editorial/creative → asymmetric.

4. Motion and scroll toolkit

Resource	Reference	Use / decision rule
MDN Scroll-driven Animations	https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Scroll-driven_animations	Native CSS scroll timelines for simple scroll-linked effects.
IntersectionObserver	https://developer.mozilla.org/en-US/docs/Web/API/IntersectionObserver	Viewport enter/leave detection without manual scroll polling.
Motion	https://motion.dev/	React animation, gestures, layout transitions and scroll-linked effects.
GSAP + ScrollTrigger	https://gsap.com/docs/v3/Plugins/ScrollTrigger/	Complex cinematic timelines, scrub, pinning, snapping.
AOS	https://michalsnik.github.io/aos/	Simple reveal-on-scroll effects.

Resource	Reference	Use / decision rule
Lenis	https://lenis.darkroom.engineering/	Smooth scrolling foundation for highly designed pages; use carefully.
Pattern	Primitive	Best use
Parallax	transform + scroll timeline / GSAP	Hero depth and image storytelling
Text reveal	clip-path/transform/opacity + view timeline / Motion / GSAP	Hero and section headings
Sticky story	position: sticky	Feature walkthroughs
Horizontal scroll	overflow-x/scroll-snap or GSAP	Portfolio/product showcase
Progress bar	scroll progress / CSS timeline	Long pages
Masonry	CSS columns/grid or library	Image/project galleries

5. APIs: connect anything without wasting hackathon time

Resource	Reference	Use / decision rule
Postman	https://www.postman.com/	Inspect APIs, test auth, debug requests and share collections.
Hoppscotch	https://hoppscotch.io/	Fast browser-based API client.
Bruno	https://www.usebruno.com/	Git-friendly API client whose collections can live in the repo.
RapidAPI	https://rapidapi.com/	Third-party API discovery; inspect quotas, pricing and terms.
OpenAPI	https://www.openapis.org/	Standard API description for contracts, docs and generated clients.
Swagger Editor	https://editor.swagger.io/	Design and validate OpenAPI specs.
Zod	https://zod.dev/	Runtime validation for TypeScript inputs/outputs.
tRPC	https://trpc.io/	Type-safe RPC when both frontend and backend are TypeScript.

Situation	Choose	Why
Third-party API	Server-side proxy / Edge Function	Protect secret keys and normalize responses.
Your TS frontend + TS backend	tRPC	Fast type-safe iteration.
Public/partner API	REST + OpenAPI	Portable, explicit contract.
Many external APIs	Adapter/service layer	Stops vendor-specific code spreading through the UI.
Webhook	Edge Function/server endpoint	Verify signature, persist event, return quickly.

Never put a secret API key in browser JavaScript. Public/publishable keys are different from secret keys; follow the provider's key model.

6. Backend choices

Resource	Reference	Use / decision rule
Supabase	https://supabase.com/	Postgres + Auth + Storage + Realtime + Edge Functions; strongest general hackathon default.
Firebase	https://firebase.google.com/	Great for mobile/consumer apps, realtime workflows and Google ecosystem.
Appwrite	https://appwrite.io/	Open-source backend platform with auth, database, storage and functions.
Convex	https://www.convex.dev/	Reactive backend/database model for TypeScript applications.

Resource	Reference	Use / decision rule
Hono	https://hono.dev/	Tiny web framework for edge/serverless APIs.
FastAPI	https://fastapi.tiangolo.com/	Python backend for AI/ML-heavy projects.
Express	https://expressjs.com/	Mature Node HTTP framework when conventional server control is needed.

Default: Supabase. Firebase for Firebase-native apps; Convex for reactive TypeScript apps; FastAPI for Python/ML; Hono for a tiny edge API.

7. Database decision system

Resource	Reference	Use / decision rule
Supabase Postgres	https://supabase.com/docs/guides/database/overview	Relational data plus the Supabase ecosystem.
Neon	https://neon.tech/	Serverless Postgres when you want database-first infrastructure.
Firestore	https://firebase.google.com/docs/firestore	Document DB for document-shaped/realtime applications.
MongoDB Atlas	https://www.mongodb.com/atlas	Flexible document data and teams already using MongoDB.
Turso	https://turso.tech/	SQLite/libSQL-oriented distributed database for lightweight/edge apps.
Upstash Redis	https://upstash.com/	Cache, rate limiting, queues and ephemeral state.

Need	Choice	Reason
Users, joins, analytics	Postgres	Relational model.
Flexible nested documents	Firestore/MongoDB	Document model.
Cache/rate limit	Redis	Fast key-value access.
Lightweight edge SQLite	Turso	Simple relational semantics.
Realtime relational app	Supabase	Postgres + Realtime + Auth together.

8. Authentication

Resource	Reference	Use / decision rule
Supabase Auth	https://supabase.com/docs/guides/auth	Email/password, magic link, OTP and social auth.
Clerk	https://clerk.com/	Polished hosted auth/user-management UX.
Auth.js	https://authjs.dev/	Flexible auth for modern JS frameworks.
Better Auth	https://www.better-auth.com/	TypeScript-first auth when you want more ownership/control.

Authentication is not authorization. A logged-in user is not automatically allowed to read every row. Use RLS/authorization rules at the data boundary.

9. AI APIs: free/cheap inference arsenal

As of August 2026, the strongest strategy is provider redundancy. Keep model/provider selection behind one adapter so a free-tier limit or outage does not kill the demo.

Resource	Reference	Use / decision rule
Gemini API pricing	https://ai.google.dev/gemini-api/docs/pricing	Selected Gemini models currently have free usage; confirm model-specific quotas.
Google AI Studio	https://aistudio.google.com/	Fast Gemini prototyping and API-key setup.
NVIDIA NIM / Build	https://build.nvidia.com/	NVIDIA currently offers free serverless inference for development on supported models.
OpenRouter pricing	https://openrouter.ai/pricing	Multi-model gateway; current free tier lists 25+ free models and 50 requests/day.
OpenRouter free router	https://openrouter.ai/openrouter/free/	Routes across available free models; model availability changes.
OpenCode	https://opencode.ai/docs	Open-source coding agent for terminal/desktop/IDE with multi-provider support.
Hugging Face Inference Providers	https://huggingface.co/docs/inference-providers/pricing	Many inference providers behind one interface; free users currently receive a small monthly credit.
Cloudflare Workers AI	https://developers.cloudflare.com/workers-ai/	Edge inference; current free allocation is 10,000 neurons/day.
Groq	https://console.groq.com/	Very fast inference when supported models fit the task.
Cerebras	https://www.cerebras.ai/	High-throughput inference useful when generation latency matters.
Together AI	https://www.together.ai/	Hosted open-model inference and tooling.
Need	First choice	Fallback
Fast multimodal prototype	Gemini	OpenRouter / NVIDIA
Many models behind one API	OpenRouter	Hugging Face
Free/open model experiments	OpenRouter free / HF	NVIDIA Build
Very low latency	Groq / Cerebras	Gemini Flash-class model
Coding agent	OpenCode + provider	Claude Code / other agent
Edge AI	Cloudflare Workers AI	Provider API behind Worker

Provider abstraction:

```
generate({ system, messages, responseFormat })  
-> { text, model, provider, usage }
```

Keep provider-specific SDK code behind this boundary.
Switching providers should be a configuration change.

10. RAG/search/AI building blocks

Resource	Reference	Use / decision rule
LlamaIndex	https://www.llamaindex.ai/	Data/RAG framework; use when document/data retrieval is central.
LangChain	https://www.langchain.com/	Broad LLM orchestration/tooling framework.
Chroma	https://www.trychroma.com/	Vector database/tooling for embeddings and semantic retrieval.
Pinecone	https://www.pinecone.io/	Managed vector database.
Qdrant	https://qdrant.tech/	Vector search engine.
Tavily	https://tavily.com/	Search API designed for AI research/agents.
Exa	https://exa.ai/	AI-oriented semantic search/retrieval API.

RAG:

documents -> chunks -> embeddings -> vector store
-> retrieve top-k -> prompt with context/citations
-> LLM -> answer

Do not build RAG just because it sounds advanced.
For small datasets, SQL/filtering may be simpler and more reliable.

11. Files, images, email and observability

Resource	Reference	Use / decision rule
Cloudinary	https://cloudinary.com/	Image/video upload, transformations, optimization and delivery.
Supabase Storage	https://supabase.com/docs/guides/storage	Object storage integrated with Auth/Postgres/RLS.
UploadThing	https://uploadthing.com/	Developer-friendly file uploads for modern web apps.
Resend	https://resend.com/	Transactional email; current free plan lists 3,000 emails/month and 100/day.
Sentry	https://sentry.io/	Error tracking and performance monitoring.

12. Deployment/infrastructure

Resource	Reference	Use / decision rule
Vercel	https://vercel.com/	Fastest default for Next.js.
Cloudflare Workers	https://developers.cloudflare.com/workers/	Edge/serverless runtime; current free plan includes 100,000 requests/day.
Cloudflare Pages	https://pages.cloudflare.com/	Static/modern web deployment.
Render	https://render.com/	Simple web services and databases; free instances are useful for testing with limitations.
Railway	https://railway.com/	Fast deployment; current free plan has \$1/month free resource credit and a time-limited trial for new users.
Docker	https://www.docker.com/	Reproducible environments for services.

Scenario	Choose	Reason
Next.js, fastest deployment	Vercel	Minimal friction.
Static frontend + edge API	Cloudflare	Network/edge integration.
Python/Node custom backend	Render/Railway	Fast service deployment.
Reproducible environment	Docker	Avoid environment drift.

Deploy early. A local project at hour 18 is not a deployed hackathon project.

13. Security: five-minute pre-demo audit

Check	Correct approach
API keys	Server/edge secrets only; never ship secret keys to browser code.
Database	RLS/authorization rules; do not trust hidden UI buttons.
Webhooks	Verify signatures before trusting payloads.

Check	Correct approach
User input	Validate with schemas such as Zod.
AI prompt injection	Treat user/retrieved text as untrusted; constrain tools and delimit data.
Uploads	Validate size/type; use managed storage; never execute uploads.
Demo data	Use synthetic data unless real data is genuinely required.

14. Agentic coding

Resource	Reference	Use / decision rule
OpenCode	https://opencode.ai/docs	Open-source coding agent with multiple providers and terminal/desktop/IDE modes.
Claude Code	https://docs.claude.com/en/docs/claude-code/overview	Repository-level coding agent.
CodeRabbit	https://www.coderabbit.ai/	AI-assisted pull-request review.
GitHub	https://github.com/	Version control, issues, PRs and CI.

AGENT LOOP

1. Acceptance criteria
2. Inspect repository
3. Plan
4. Implement one vertical slice
5. Build + tests
6. Inspect git diff
7. Review
8. Fix
9. Commit
10. Repeat

GSD/Ralph-style workflows are useful patterns, but third-party implementations vary. Treat them as optional layers, not magic switches.

15. Hackathon architecture recipes

Project	Recommended stack	Why
AI SaaS/chatbot	Next.js + shadcn + Supabase + Gemini/OpenRouter	Fast auth, DB and AI.
RAG/document assistant	Next.js + Storage + vector search + Gemini/OpenRouter	Upload → retrieve → answer → citations.
Real-time collaboration	Next.js + Supabase Realtime / Convex	Realtime state without hand-built WebSockets.
ML-heavy project	Next.js + FastAPI + Postgres + AI provider	Python stays where ML lives.
Data dashboard	Next.js + Supabase + Chart.js + Motion	Fast CRUD + polished analytics.
Creative portfolio	Next.js + Tailwind + Motion/GSAP + Stitch/UiVerse	Visual polish is the product.

16. Winning frontend formula

1 hero → 1 primary action → 1 proof/result → 3 supporting features → 1 credibility section → 1 final CTA.

For an application: clear navigation → obvious primary task → immediate feedback → loading/error/empty states → useful result → history/details.

17. What to avoid

- Microservices because the diagram looks impressive.
- Five databases when Postgres would do.
- RAG where SQL would solve it.
- Agents where deterministic code is safer.
- Twelve animation libraries.
- Secret keys in frontend code.
- Building auth from scratch.
- Polishing the hero before the core flow works.
- Depending on one free AI provider.
- Deploying only at the end.

18. 24-hour execution plan

Time	Target
0-1h	Problem, user, demo sentence, acceptance criteria, architecture.
1-2h	Wireframe/design direction; initialize repo; deploy skeleton.
2-6h	One complete end-to-end vertical slice.
6-10h	Database/auth/API/AI integration; error states.
10-14h	Responsive UI + visual system + core animations.
14-18h	Testing, fallback/mock mode, performance/security pass.
18-21h	Polish typography, spacing, micro-interactions and demo data.
21-23h	Demo script, screenshots/video, README and architecture diagram.
23-24h	Freeze, deploy, clean-browser test. Sleep if humanly possible.

19. Final decision cheat sheet

If you need...	Reach for...
Beautiful components	shadcn/ui + Ulverse + Magic UI/Aceternity
Accessible primitives	Radix / Headless UI
Simple scroll reveals	AOS / IntersectionObserver
Cinematic scroll	GSAP ScrollTrigger
React motion	Motion
Auth + DB + storage + realtime	Supabase
Mobile/Google ecosystem	Firebase
Python AI backend	FastAPI
Tiny edge API	Hono
SQL	Postgres / Supabase / Neon
Cache/rate limit	Upstash Redis
AI fallback	OpenRouter
Gemini experimentation	Gemini API / AI Studio
NVIDIA inference	NVIDIA Build / NIM
Multi-provider coding agent	OpenCode
Email	Resend
Files/media	Supabase Storage / Cloudinary
Next.js deployment	Vercel
Custom backend deployment	Render / Railway
Edge deployment	Cloudflare Workers
API debugging	Postman / Hoppscotch / Bruno

Final principle

Winning hackathons is not about using the most technologies. Choose the smallest reliable stack that demonstrates the most value. Make the product understandable, the happy path flawless, the UI memorable, the architecture defensible, and the demo resilient to third-party failures.

Expanded continuation of the resource vault created from your supplied reels. Future reels can be folded into the same playbook.